

Building Your First Android Greeting App

Java/Kotlin, Android Studio & SDK Fundamentals

Level	Beginner	Estimated Time	45-60 minutes
Mode	Hands-on Practical	Platform	Android Studio

1. Aim

To create a simple Android “Greeting App” that accepts a user’s name and displays a personalized greeting. The practical introduces Android Studio project setup, XML-based UI design, event handling in Java/Kotlin, basic debugging, and Git initialization.

2. Learning Objectives

- Set up a new Android project and understand the basic project structure.
- Design a simple user interface using XML with EditText, Button, and TextView.
- Handle a button click in Kotlin or Java and read user input safely.
- Update a TextView dynamically based on the entered name.
- Run and debug the app using an Android emulator or physical device.
- Initialize a local Git repository and create the first commit.

3. Software / Requirements

Requirement	Recommended Setup
Android Studio	Latest stable version
Android SDK	API 24 or above
Language	Kotlin preferred; Java alternative provided
Device	Android Emulator or USB-debug enabled Android phone

4. Practical Scenario

You are building the first screen of a beginner Android app. The screen should ask the learner to enter a name. When the learner taps “Greet Me”, the app should show “Hello, <name>! Welcome to Android.” If the field is empty, the app should show a short validation message.

5. Procedure

A. Create a New Android Project

1. Open Android Studio and choose New Project.
2. Select Empty Views Activity (or the closest XML-based empty activity template).
3. Set the app name to GreetingApp.
4. Choose Kotlin or Java as the language and set the minimum SDK to API 24 or above.
5. Click Finish and wait for Gradle sync to complete.

B. Design the User Interface

Open app > res > layout > activity_main.xml and create a simple vertical layout with a heading, a name input, a button, and a result TextView.

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp"
    android:gravity="center_horizontal">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Greeting App"
        android:textSize="24sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:hint="Enter your name"
        android:inputType="textPersonName" />

    <Button
        android:id="@+id/btnGreet"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Greet Me" />

    <TextView
        android:id="@+id/tvGreeting"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:textSize="18sp" />
</LinearLayout>
```

C. Add the App Logic - Kotlin

MainActivity.kt

```
package com.example.greetingapp

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)

val nameInput = findViewById<EditText>(R.id.etName)
val greetButton = findViewById<Button>(R.id.btnGreet)
val greetingText = findViewById<TextView>(R.id.tvGreeting)

greetButton.setOnClickListener {
    val name = nameInput.text.toString().trim()

    if (name.isEmpty()) {
        nameInput.error = "Please enter your name"
    } else {
        greetingText.text = "Hello, $name! Welcome to Android."
    }
}
}
}

```

Java Alternative

MainActivity.java

```

package com.example.greetingapp;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        EditText nameInput = findViewById(R.id.etName);
        Button greetButton = findViewById(R.id.btnGreet);
        TextView greetingText = findViewById(R.id.tvGreeting);

        greetButton.setOnClickListener(v -> {
            String name = nameInput.getText().toString().trim();

            if (name.isEmpty()) {
                nameInput.setError("Please enter your name");
            } else {
                greetingText.setText("Hello, " + name + "! Welcome to Android.");
            }
        });
    }
}

```

D. Run and Test the App

1. Start an Android emulator or connect a physical Android device with USB debugging enabled.
2. Click Run in Android Studio and select the target device.
3. Enter a name such as Saba and tap Greet Me.
4. Confirm that the greeting appears correctly.
5. Clear the name field and tap Greet Me again. Confirm that a validation error is shown.

6. Test Cases / Validation Checklist

Test Case	Input / Action	Expected Result	Status
TC-01	Enter “Alex” and tap Greet Me	Hello, Alex! Welcome to Android.	☐ Pass ☐ Fail
TC-02	Leave name empty and tap Greet Me	Validation error is displayed	☐ Pass ☐ Fail
TC-03	Enter name with spaces around it	Spaces are trimmed in output	☐ Pass ☐ Fail
TC-04	Rotate device after greeting	App should remain usable	☐ Pass ☐ Fail

7. Basic Debugging Exercise

Place a breakpoint inside the button click listener. Run the app in Debug mode, enter a name, and tap the button. Inspect the value of the variable name. Then continue execution and verify the result on screen.

- Observe Logcat for runtime errors.
- If the app crashes, identify the exception and the line number.
- Check that all XML IDs used in code match the IDs in activity_main.xml.

8. Git Version Control Task

From the project folder, initialize Git and create an initial commit after the app works successfully.

```
git init
git add .
git commit -m "Create first Android greeting app"
```

9. Expected Output

The final screen contains a title, a text field for the user’s name, a “Greet Me” button, and a result area. Example: entering “Alex” should display “Hello, Alex! Welcome to Android.”

10. Student Challenge - Optional Enhancement

- Add a Reset button that clears the input and greeting.
- Change the greeting based on time of day: Good Morning / Good Afternoon / Good Evening.
- Add a second input for the learner’s course and include it in the greeting.
- Use View Binding instead of findViewById.
- Preserve the greeting after screen rotation using savedInstanceState or ViewModel.

11. Viva / Review Questions

1. What is the role of AndroidManifest.xml in an Android project?

2. What is the difference between an Activity and a layout XML file?
3. Why is trim() used before validating the name?
4. What does setOnClickListener do?
5. How does findViewById connect Kotlin/Java code to XML views?
6. What is the purpose of the Android SDK?
7. What is Logcat and when would you use it?
8. Why is version control useful in an Android project?

12. Submission Requirements

- Working Android Studio project folder or Git repository link.
- Screenshot of the app showing a successful greeting.
- Screenshot showing the empty-name validation.
- Short note (3-5 lines) describing one issue encountered and how it was fixed.

13. Result

The Android Greeting App was created and tested successfully using Android Studio. The practical demonstrated XML UI creation, Java/Kotlin event handling, user-input validation, debugging, and basic Git version control.

Student Name:	_____
Date / Faculty Sign:	_____